Advanced Graphs

Bridges & articulation points, biconnected components, 2-SAT, Euler tours

Learning goals

- Derive the **lowlink** criteria for bridges and cut vertices — and prove them
- Build the **bridge tree** and the **block-cut tree**, and reduce problems to trees
- Model boolean constraints as **2-SAT**, prove the SCC criterion, extract an assignment
- Decide Euler path/circuit existence and construct one with **Hierholzer** in O(m)
- Handle multi-edges, self-loops, disconnected graphs, and deep recursion

Prereqs: Year weeks 16, 21 (DFS, SCC), 20/22 (trees, LCA)

Motivating problem

> Undirected connected graph, n, m ≤ $3 \cdot 10^5$. A boss is placed on every edge whose removal disconnects s from t. Choose s, t to **maximize** the number of bosses.
> (Codeforces 1000E *We Need More Bosses*)

- Edges separating s and t are **bridges** on every s–t path
- Contract each 2-edge-connected component → a tree whose edges are the bridges
- Answer = **diameter** of that tree

Today's theme: *compress the graph to a tree, then use tree techniques.*

DFS tree and lowlink

Run DFS; record entry time `tin[v]`.

`low[v]` = min `tin` reachable from v's subtree using tree edges down and **at most one back edge** up.

```
low[v] = min( tin[v],
              tin[u]  for back edges v–u,
              low[c]  for tree children c )
```

In an undirected DFS there are only **tree edges** and **back edges** (no cross edges)

—

this is what makes lowlink work.

Bridge and cut-vertex criteria

Tree edge (v → c):

- **Bridge** ⇔ `low[c] > tin[v]`
  (nothing in c's subtree reaches v or above without this edge)
- v is a **cut vertex** (non-root) ⇔ some child c has `low[c] ≥ tin[v]`
  (c's subtree cannot reach a proper ancestor of v)
- Root is a cut vertex ⇔ it has **≥ 2 DFS children**

Back edges are never bridges (they lie on a cycle with tree edges).

Proof sketch (bridge criterion)

(⇐) If `low[c] > tin[v]`, every edge leaving subtree(c) other than (v, c) ends inside subtree(c) — back edges only go to ancestors, and none reaches tin ≤ tin[v]. Deleting (v, c) separates subtree(c).

(⇒) If `low[c] ≤ tin[v]`, some back edge from subtree(c) goes to v or an ancestor u of v. Then u ⇝ v → c ⇝ (back-edge source) → u is a cycle through (v, c), so it is not a bridge. ∎

The cut-vertex proof is identical with "≥" because reaching v itself does not help once v is deleted.

Multi-edges: skip the edge, not the parent

```
for (auto [to, id] : adj[v]) {
    if (id == parentEdge) continue;   // NOT: if (to == parent) continue;
    ...
}
```

Two parallel edges u–v form a cycle: neither is a bridge.

Skipping by vertex would wrongly report the bridge.

Self-loops: never bridges, never matter for cut vertices — usually drop them.

Lowlink DFS with edge stack

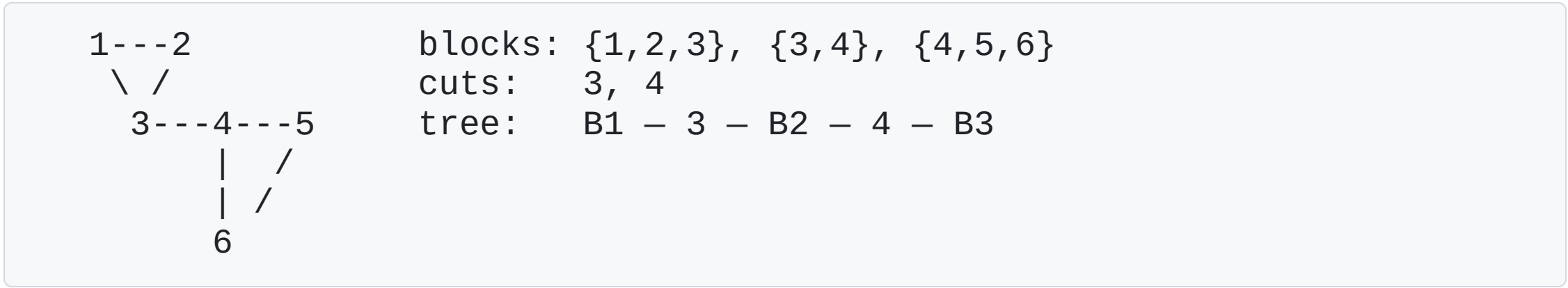
```
void dfs(int v, int parentEdge) {
    tin[v] = low[v] = timer++;
    int children = 0;
    for (auto [to, id] : adj[v]) {
        if (id == parentEdge) continue;
        if (tin[to] != -1) {                          // back edge
            if (tin[to] < tin[v]) edgeStack.push_back(id);
            low[v] = min(low[v], tin[to]);
            continue;
        }
        edgeStack.push_back(id);
        dfs(to, id); children++;
        low[v] = min(low[v], low[to]);
        if (low[to] > tin[v]) isBridge[id] = true;
        if (low[to] >= tin[v]) {                      // pop one block
            if (parentEdge != -1) isCut[v] = true;
            vector<int> comp;
            while (true) { int e = edgeStack.back(); edgeStack.pop_back();
                           comp.push_back(e); if (e == id) break; }
            bccs.push_back(comp);
        }
    }
    if (parentEdge == -1 && children >= 2) isCut[v] = true;
}
```

Two kinds of biconnectivity

	2-edge-connected	2-vertex-connected (blocks)
Separated by	bridges	cut vertices
Partition of	**vertices**	**edges** (vertices can be shared)
Compute	components after deleting bridges	edge stack, pop when `low[c] ≥ tin[v]`
Compressed graph	**bridge tree**	**block-cut tree**
Path property	u–v edge-disjoint paths ≥ 2 ⇔ same comp	2 vertex-disjoint paths ⇔ same block

The block-cut tree

Nodes: one per block, one per cut vertex. Edge (block B, cut c) if c ∈ B.



- Vertices every u–v path must visit = cut vertices on the tree path
- Combine with LCA / HLD (Lectures 4, Year 22): CF 487E *Tourists* (min price over all vertices on any simple u–v path)

Robbins' theorem & orientations

Theorem. An undirected graph has a strongly connected orientation ⇔ it is connected and **bridgeless**.

Construction: orient tree edges **down**, back edges **up** (in DFS order). Every vertex reaches the root via its low-link back edge chain, and the root reaches all.

- CSES 2177 *Strongly Connected Edges*, CF 118E *Bertown roads*
- CF 732F *Tourist Reform*: maximize min reachability ⇒ orient the **largest** 2-edge-connected component strongly, point bridges toward it

2-SAT: implication graph

Variables $x_1..x_n$, clauses $(a \vee b)$ with literals a, b.

$(a \vee b) \equiv (\neg a \to b) \wedge (\neg b \to a)$ — add both edges. 2n nodes.

Theorem. Satisfiable $\Leftrightarrow$ no variable has x and ¬x in the same SCC.

Assignment. Let `comp` be SCC indices in **topological** order of the condensation.
Set x = true iff comp(x) comes **after** comp(¬x).
(With Tarjan, SCC ids are *reverse* topological: x = true iff `id[x] < id[¬x]`.)

Why the assignment works

Suppose the rule produces a violated clause $(a \vee b)$: a and b both false.

- a false $\Rightarrow \neg a$ is topologically after a; b false $\Rightarrow \neg b$ after b
- Edge $\neg a \to b \Rightarrow$ b is not before $\neg a$; edge $\neg b \to a \Rightarrow$ a not before $\neg b$
- Chain: $a \prec \neg a \preceq b \prec \neg b \preceq a$ — contradiction (strict order cycle) ∎

The implication graph is **skew-symmetric** ($u \to v \iff \neg v \to \neg u$), which makes the SCC of $\neg x$ the "mirror" of the SCC of x.

2-SAT code

```
struct TwoSat {
    int n; vector<vector<int>> g;            // literal 2x = x, 2x+1 = ¬x
    TwoSat(int n) : n(n), g(2 * n) {}
    void addOr(int a, int b) { g[a ^ 1].push_back(b); g[b ^ 1].push_back(a); }
    void addImplies(int a, int b) { addOr(a ^ 1, b); }
    void forceTrue(int a) { addOr(a, a); }
    // tarjan(...) fills comp[] in reverse topological order
    bool solve() {
        /* run Tarjan on all 2n literals */
        for (int x = 0; x < n; x++) {
            if (comp[2 * x] == comp[2 * x + 1]) return false;
            value[x] = comp[2 * x] < comp[2 * x + 1];
        }
        return true;
    }
};
```

Full version with Tarjan in the notes. O(n + m).

2-SAT modeling cheat sheet

Constraint	Clauses
a must be true	$(a \vee a)$
$a \to b$	$(\neg a \vee b)$
$a \neq b$ (xor)	$(a \vee b), (\neg a \vee \neg b)$
$a = b$	$(\neg a \vee b), (a \vee \neg b)$
at most one of $a_1..a_k$	prefix variables $p_i = a_1 \vee \ldots \vee a_i$: $O(k)$ clauses
interval choices "pick ≥ / ≤ threshold"	variables "value ≥ t" with chain implications

Signals in a statement: each item has **two options**, conflicts are **pairwise**.

CF 776D *The Door Problem*: each door controlled by 2 switches → xor/equality

Euler paths & circuits

Graph	Circuit exists ⇔	Path exists ⇔
Undirected	all degrees even	0 or 2 odd-degree vertices
Directed	in = out everywhere	≤ 1 vertex out−in = 1, ≤ 1 with in−out = 1, rest equal

…**plus**: all edges in one (weakly) connected component.
Check connectivity by verifying the produced trail has **m + 1** vertices.

Start at an odd vertex (undirected) / the out−in = 1 vertex (directed).

Hierholzer's algorithm

Walk unused edges until stuck; when stuck, emit the vertex and back up.

Stuck at v ⇒ all of v's edges are used; emitted vertices form the trail in reverse.

```
vector<int> ptr(n, 0), path, st = {start};
vector<bool> used(m, false);
while (!st.empty()) {
    int v = st.back();
    while (ptr[v] < (int)adj[v].size() && used[adj[v][ptr[v]].second]) ptr[v]++;
    if (ptr[v] == (int)adj[v].size()) { path.push_back(v); st.pop_back(); continue; }
    auto [to, id] = adj[v][ptr[v]++];
    used[id] = true;
    st.push_back(to);
}
if ((int)path.size() != m + 1) /* not connected */;
reverse(path.begin(), path.end());
```

`ptr[v]` makes it O(n + m): each adjacency entry is skipped once.

Euler applications

- **De Bruijn sequence** (CSES 1692): nodes = (k–1)-length words, edges = k-length words; Euler circuit
- **Word chains / reconstruct string from all 3-substrings** (CF 508D *Tanya and Password*)
- **Make degrees balanced**: connect all odd vertices to a dummy node, take an Euler circuit, orient along it ⇒ every even-degree vertex gets in = out (CF 723E *One-Way Reform*)
- **Chinese postman** (undirected): add a min-weight perfect matching of odd vertices
- Domino / edge-covering-walk puzzles: think "vertices are values, edges are pieces"

Complexity summary

Algorithm	Time	Memory
Lowlink (bridges, cuts, blocks)	O(n + m)	O(n + m)
Bridge tree / block-cut tree	O(n + m)	O(n + m)
2-SAT (Tarjan / Kosaraju)	O(n + m)	O(n + m)
Hierholzer	O(n + m)	O(n + m)

Recursion depth up to n: with n = 10^6 on a path, use an **iterative** DFS or raise the stack limit (some judges allow `ulimit -s` /linker flags; many do not).

Common pitfalls

- Skipping the parent **vertex** instead of the parent **edge** (multi-edges)
- Root cut-vertex rule forgotten (root with one child is *not* a cut vertex)
- Using `low[to]` from an already-finished vertex in **directed** Tarjan without `onStack`
- 2-SAT: adding only one of the two implications of a clause
- 2-SAT: inverted assignment rule (check Tarjan vs Kosaraju numbering!)
- Euler: forgetting the connectivity check, or starting at the wrong vertex
- Euler on undirected graph: marking only one direction of an edge as used

Practice problems

- CSES 2076 *Necessary Roads*, 2077 *Necessary Cities*, 2177 *Strongly Connected Edges*
- CSES 1684 *Giant Pizza*, 1691 *Mail Delivery*, 1693 *Teleporters Path*, 1692 *De Bruijn Sequence*
- Codeforces 1000E *We Need More Bosses*, 118E *Bertown roads*, 732F *Tourist Reform*
- Codeforces 776D *The Door Problem*, 228E *The Road to Berland is Paved With Good Intentions*
- Codeforces 1215F *Radio Stations*, 508D *Tanya and Password*, 723E *One-Way Reform*
- Codeforces 962F *Simple Cycles Edges*, stretch: 487E *Tourists*
- Library Checker *Two-Edge-Connected Components*, *Biconnected*

Summary

- Lowlink gives bridges/cut vertices/blocks in one DFS; skip the parent **edge**
- Compress to the **bridge tree** or **block-cut tree**, then solve on a tree
- 2-SAT = implication graph + SCC; assignment from topological order
- Euler: degree parity + connectivity; Hierholzer builds it in linear time
- Next lecture: mock national championship